



DOCUMENTATION TECHNIQUE

Installation

Mettre en place l'environnement de développement

Sommaire

1. Prérequis

2. Récupération et dépendances

3. Variables d'environnement

4. Services locaux

5. Amorçage du contenu

Import des visuels du client

6. Lancement

7. Vérifications

8. Modifier le modèle de contenu

9. Problèmes fréquents

1. Prérequis

Outil	Version	Vérification
Node.js	≥ 20.9 (testé sur 24)	<code>node -v</code>
npm	≥ 10	<code>npm -v</code>
Docker	récent	<code>docker --version</code>
Git	récent	<code>git --version</code>

PostgreSQL peut être installé nativement, mais `docker compose` fournit une base déjà configurée sur un port qui n'entre pas en conflit avec d'autres projets.

2. Récupération et dépendances

```
git clone <url-du-depot> romial-kenmogne
cd romial-kenmogne
npm install
```

3. Variables d'environnement

```
cp .env.example .env
```

Puis, dans `.env`, générer un secret unique :

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

et le coller dans `PAYLOAD_SECRET`.

Variables essentielles :

Variable	Rôle	Valeur locale
<code>NEXT_PUBLIC_SITE_URL</code>	Base des URL canoniques, hreflang, sitemap, e-mails	<code>http://localhost:4313</code>
<code>DATABASE_URI</code>	Connexion PostgreSQL	<code>postgres://romial:romial@localhost:5437/romial_website</code>
<code>PAYLOAD_SECRET</code>	Signature des sessions CMS	chaîne aléatoire ≥ 32 caractères
<code>CMS_ENABLED</code>	<code>false</code> pour servir le contenu de démarrage sans base	<code>true</code>
<code>EMAIL_ENABLED</code>	Active l'envoi réel d'e-mails	<code>true</code> avec MailHog
<code>SMTP_*</code> , <code>EMAIL_FROM</code> , <code>EMAIL_TO</code>	Compte SMTP et destinataires	MailHog sur <code>localhost:1026</code>
<code>CONTACT_RATE_LIMIT</code>	Demandes maximum par IP et par fenêtre	<code>50</code> en local, 5 en production
<code>NEXT_PUBLIC_ANALYTICS_*</code>	Umami / Plausible, ou Google Analytics 4 (<code>google</code> + <code>G-...</code> , avec bannière)	vide = désactivé
<code>SEED_ADMIN_EMAIL</code> / <code>SEED_ADMIN_PASSWORD</code>	Premier compte administrateur	à changer après la première connexion

`.env` n'est **jamais** versionné (`.gitignore`).

4. Services locaux

```
docker compose up -d
```

Service	Port	Usage
PostgreSQL 16	5437	Base de données
MailHog (SMTP)	1026	Réception des e-mails de test
MailHog (interface)	8026	http://localhost:8026

Vérification : `docker compose ps` doit afficher les deux conteneurs *healthy*.

5. Amorçage du contenu

```
npm run seed
```

Le script est **idempotent** (relançable sans créer de doublons). Il crée :

- le premier compte administrateur (`SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`) ;
- 9 catégories RK Insights ;
- 8 domaines d'expertise rédigés dans les trois langues ;
- 3 fiches d'expérience **d'exemple** (organisations entre crochets, à remplacer) ;
- 3 articles RK Insights **d'exemple** ;
- la fiche du livre « Réussir son premier achat immobilier en Europe » ;
- les 5 activités de l'écosystème (RK Business Consulting, RK IMMO-FINANZ, Kenmogne Strategic Publishing, KAILI Institut, KAILI Event) avec des descriptions à compléter ;
- 5 pages légales à l'état de **trame juridique** ;
- les réglages du site, de la page d'accueil et de la page À propos.

Tout ce qui doit être validé par le commanditaire est marqué « **Contenu d'exemple** » dans le CMS et affiché comme tel sur le site public.

Import des visuels du client

Les photographies et la couverture du livre ne sont **pas** dans le dépôt Git (données personnelles / visuels client) : elles vivent dans la médiathèque du CMS et ses sauvegardes. Pour les (ré)importer dans un nouvel environnement :

```
npm run import:assets -- "<dossier contenant RK.jpeg, RK1.jpeg, RK2.jpeg>" "<image de couverture>"
```

Le script est idempotent : il réutilise les fichiers déjà présents, renseigne les textes alternatifs en FR/DE/EN, associe la photo du hero, le portrait de la page À propos et la couverture du livre, et n'écrase jamais une biographie déjà modifiée dans le CMS. Les métadonnées EXIF (dont la géolocalisation) sont supprimées à l'import.

6. Lancement

```
npm run dev
```

Adresse	Contenu
http://localhost:4313	Redirection vers la langue du navigateur
http://localhost:4313/fr · /de · /en	Site public
http://localhost:4313/admin	Administration CMS
http://localhost:4313/sitemap.xml	Sitemap
http://localhost:8026	E-mails capturés

Première connexion à l'administration : identifiants du seed, puis **changement immédiat du mot de passe** (menu compte → *Change password*).

7. Vérifications

```
npm run verify      # format (src, tests, docs) + types + lint + tests unitaires
npm run test:e2e    # tests end-to-end (démarré le serveur si besoin)
```

Intégration continue : à chaque push sur `main` et à chaque pull request, GitHub Actions (`.github/workflows/ci.yml`) vérifie le formatage, TypeScript, le lint, les tests unitaires et le build de production, sur Node 22, sans base de données ni secret (comme la construction de l'image Docker).

Un second job (`e2e`) lance les tests end-to-end sur Chromium et en largeur mobile, contre le build de production : il démarre une base PostgreSQL et un MailHog **jetables**, crée le schéma avec les migrations (`npm run migrate`), charge le contenu de départ (`npm run seed`), construit le site puis exécute `npx playwright test`. Les valeurs de ce job (mot de passe de la base, `PAYLOAD_SECRET`, compte administrateur) n'existent que dans ce job et ne servent à aucun site réel. En cas d'échec, le rapport Playwright est joint au run pendant 7 jours. Firefox et WebKit restent lancés en local (`E2E_ALL_BROWSERS=1`) avant chaque livraison.

Image Docker : après une CI verte sur `main`, le workflow *Docker image* (`.github/workflows/docker.yml`) construit l'image de production. Il la publie sur `ghcr.io/carine2107/rk-portfolio` dès que la variable de dépôt `NEXT_PUBLIC_SITE_URL` est renseignée (voir `DEPLOIEMENT.md` §3). Aucun déploiement automatique n'est configuré.

Captures d'écran de recette (3 langues × 2 thèmes × 5 largeurs) :

```
node tests/visual/capture.mjs test-results/visual
```

8. Modifier le modèle de contenu

Après toute modification dans `src/payload/` :

```
npm run generate:types      # met à jour src/payload-types.ts
npm run generate:importmap  # si un composant d'admin personnalisé est ajouté
npm run seed:reply-templates # modèles de réponse d'exemple (seulement ceux qui manquent)
```

En développement, le schéma de base est synchronisé automatiquement (`push: true`). En production, voir `DEPLOIEMENT.md` (Déploiement_et_exploitation.pdf).

9. Problèmes fréquents

Symptôme	Cause probable	Solution
<code>EADDRINUSE :4313</code>	Port déjà pris	Changer le port dans <code>package.json</code> et <code>NEXT_PUBLIC_SITE_URL</code>
Le site affiche « contenu de démarrage »	Base inaccessible	<code>docker compose up -d</code> , vérifier <code>DATABASE_URI</code>
Aucun e-mail reçu	<code>EMAIL_ENABLED=false</code> ou SMTP invalide	Vérifier les variables <code>SMTP_*</code> , consulter les logs serveur
<code>PAYLOAD_SECRET</code> refusé au démarrage	Secret manquant ou trop court en production	Générer un secret de 48 octets
Formulaire : « Trop de demandes »	Limitation de débit atteinte	Attendre la fenêtre, ou augmenter <code>CONTACT_RATE_LIMIT</code> en local
<code>format:check</code> signale tous les fichiers (Windows)	Fins de ligne CRLF au clonage (<code>core.autocrlf</code>)	<code>.gitattributes</code> impose LF. Après avoir commité ou mis de côté (<code>git stash</code>) vos modifications — la commande suivante efface tout changement non enregistré : <code>git rm --cached -r . && git reset --hard</code>
Script Payload bloqué sur « Pulling schema from database... » (<code>ECONNRESET</code>)	Synchronisation automatique du schéma interrompue	Relancer ; si le schéma est déjà à jour, préfixer la commande par <code>PAYLOAD_MIGRATING=true</code> (saute cette synchronisation, n'applique aucune migration)